

Звіт з лабораторної роботи №2

Тема: Контейнеризація

Виконав: Комар Тимофій Олегович, ІМ-43

Мета роботи

Практичне закріплення знань з основ контейнеризації, використання Docker для пакування застосунків та Docker Compose для автоматизованого запуску багатокомпонентних програмних систем.

1. Дослідницька частина

1.1. Python Application: Оптимізація шарів (Layers)

Для експерименту використовувався базовий образ `python:3.11-bookworm` на базі Debian.

- **Неоптимізована збірка:** Dockerfile був написаний так, що спочатку копіювався весь код, а потім встановлювалися залежності. Час первинної збірки склав **1 хвилину 55 секунд**. При зміні одного рядка в коді час повторної збірки залишився таким самим, оскільки кеш Docker був інвалідований, і залежності завантажувалися заново.
- **Оптимізована збірка:** Порядок команд у Dockerfile було змінено: спочатку копіювався лише `requirements.txt` і виконувався `pip install`, а потім копіювався код застосунку. Завдяки ефективному використанню кешу шарів, після зміни вихідного коду час повторної збірки скоротився до **1.2 секунди**.

1.2. Python Application: Alpine Linux та залежності (Numpy)

Для порівняння розмірів базовий образ було змінено на надлегкий `python:3.11-alpine`, а в проєкт додано C-залежність `numpy`.

- **Розмір образу:** Використання Alpine дозволило суттєво зменшити загальний розмір фінального образу — до **271 МБ**, порівняно з ~1 ГБ для образу на базі `bookworm`.
- **Час збірки:** Очікувалося значне збільшення часу збірки через необхідність компіляції `numpy` з вихідного коду C під бібліотеку `musl`. Проте експеримент показав час збірки всього **41 секунду**. Це пов'язано з розвитком стандарту `musllinux` у сучасній екосистемі Python, що дозволяє менеджеру пакетів `pip` завантажувати готові оптимізовані `wheels` навіть для Alpine Linux.

1.3. Musl (Alpine) vs glibc (Ubuntu): Робота з DNS

Експеримент перевіряв поведінку різних системних бібліотек при резолвінгу DNS-імен у закритій Docker-мережі з використанням пошукового домену.

- **Ubuntu (glibc):** Успішно розрезолвив адресу (10.0.0.50). Резолвер побачив коротке ім'я myservice.internal, автоматично додав пошуковий домен, перетворивши його на myservice.internal.corp, і знайшов ціль.
- **Alpine (musl):** Не зміг розрезолвити адресу та повернув порожній результат. Через специфіку резолвера musl, якщо в запиті вже присутня крапка (.), він вважає його повним доменним ім'ям і повністю ігнорує директиву search.
- **Висновок:** Використання Alpine у великих мікросервісних інфраструктурах може призводити до неочевидних багів у service discovery, коли сервіси не можуть взаємодіяти між собою за короткими внутрішніми іменами через ігнорування search-доменів.

1.4. Golang Application: Багатоетапна збірка (Multi-stage builds)

Було проведено тестування пакування скомпільованого застосунку на мові Go.

- **Звичайна збірка:** Образ на базі стандартного golang:1.21 має розмір **1.3 ГБ**. До образу потрапляє компілятор, системні утиліти ОС та вихідний код, що є надлишковим для запуску бінарного файлу і створює загрози безпеці.
- **Multi-stage збірка:** Збірка була розбита на два етапи: спочатку код компілювався у статичний бінарник з вимкненим CGO, а потім переносився у порожній образ FROM scratch. Розмір фінального образу склав усього **16.2 МБ**.
- **Висновок щодо зручності:** Образ scratch є ідеальним з точки зору економії ресурсів пам'яті. Однак він абсолютно незручний для дебагу, оскільки не містить командної оболонки, що унеможливорює підключення до контейнера через docker exec. Альтернативою для продакшену є використання образів distroless, які зберігають мінімалізм, але додають базові компоненти.

2. Практична частина

Для автоматизованого запуску застосунку, розробленого в рамках Лабораторної роботи №1, було впроваджено технологію Docker Compose.

Виконані кроки:

1. **Створення Dockerfile:** Написано інструкцію для збірки образу Python-застосунку на базі python:3.11-slim із встановленням залежностей Flask та PyMySQL.
2. **Налаштування Nginx:** Створено локальний файл nginx.conf із правилами reverse проху та заборonoю доступу до ендпоінту /health, який прокидається у контейнер через volumes.
3. **Конфігурація docker-compose.yml:**
 - Описано 3 сервіси: web, nginx та db MariaDB 10.11.
 - Усі сервіси ізольовано у власній віртуальній мережі app_network.

- Для бази даних створено іменований volume (db_data), що забезпечує збереження інформації навіть після видалення контейнерів чи перезавантаження системи.
- Автоматизовано процес міграції: при старті сервісу web виконується команда для запуску скрипта міграції, після чого стартує основний застосунок.

Посилання на репозиторій із виконаною практичною частиною:

https://github.com/SkVaYeR228/Lab1_SDTCS

Загальні висновки та рекомендації

Проведені дослідження довели необхідність свідомого підходу до створення Docker-образів:

1. **Оптимізація кешу:** Завжди слід розміщувати інструкції встановлення залежностей до копіювання вихідного коду для прискорення CI/CD пайплайнів.
2. **Вибір базового образу:** Використання Alpine Linux виправдане для зменшення об'єму пам'яті, проте слід зважати на можливі проблеми з мережевим резолвінгом musl libc та швидкістю компіляції C-залежностей.
3. **Багатоетапна збірка:** Для компільованих мов (Golang, C++) обов'язковим стандартом є використання Multi-stage builds у комбінації з scratch або distroless для створення максимально компактних та безпечних артефактів.